Práctica: Construcción Procedural de un Escenario 3D

Objetivo: Implementar transformaciones tridimensionales (traslación y escalamiento) y modelos de color mediante scripting en Blender.

Paso 1: Limpieza del Entorno y Preparación

Antes de construir, debemos asegurar que la escena esté vacía. Esto enseña a los alumnos la importancia de la gestión de objetos en memoria.

- **Lógica:** Seleccionar todo y borrar.

```
import bpy

# Limpiamos la escena para empezar de cero
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete()
```

Paso 2: Definición de Materiales (Modelos de Color)

Según el tema **1.4 (Modelos del color)** , utilizaremos el modelo **RGB** para dar vida al escenario.

- **Lógica:** Crear una función que genere materiales para no repetir código (fomentando el paradigma orientado a objetos).

```
def crear_material(nombre, r, g, b):
    # Creamos un nuevo material en la base de datos de Blender
    mat = bpy.data.materials.new(name=nombre)
    # Definimos el color difuso (RGBA)
    mat.diffuse_color = (r, g, b, 1.0)
    return mat

# Definimos dos colores para nuestro escenario
mat_base = crear_material("GrisOscuro", 0.1, 0.1, 0.1)
mat_acento = crear_material("Neon", 0.0, 0.8, 1.0) # Un azul cian
```

Paso 3: Construcción de Paredes con Ciclos (Transformaciones)

Aquí aplicamos el **Tema 3.3.1 (Traslación)**. Usaremos un ciclo `for` para automatizar la creación de un pasillo.

- **Lógica:** Iterar para colocar cubos en el eje Y, alternando materiales según el índice del ciclo.

```
largo = 10  # Cantidad de bloques

for i in range(largo):
    # --- Pared Izquierda ---
    # Traslación: colocamos el cubo en (x=-3, y=i*2, z=1)
    bpy.ops.mesh.primitive_cube_add(location=(-3, i * 2, 1))
    bloque_izq = bpy.context.active_object

    # Aplicamos lógica de color (Relleno de polígonos - Tema 4.1)
    if i % 2 == 0:
        bloque_izq.data.materials.append(mat_base)
    else:
        bloque_izq.data.materials.append(mat_acento)
        # Escalamiento (Tema 3.3.2): hacemos que los bloques de color sean
        bloque_izq.scale.z = 1.5
```

Paso 4: Completando la Geometría (Simetría y Suelo)

Para que parezca un videojuego, cerramos el pasillo y agregamos una superficie base.

- **Lógica:** Replicar la pared en el lado positivo del eje X y añadir un plano escalado para el suelo.

```
for i in range(largo):
    # --- Pared Derecha ---
    bpy.ops.mesh.primitive_cube_add(location=(3, i * 2, 1))
    bloque_der = bpy.context.active_object
    bloque_der.data.materials.append(mat_base)

# --- Suelo ---
# Creamos un plano y lo escalamos para cubrir todo el pasillo
bpy.ops.mesh.primitive_plane_add(location=(0, (largo - 1), 0))
suelo = bpy.context.active_object
suelo.scale.x = 4
suelo.scale.y = largo + 2
```

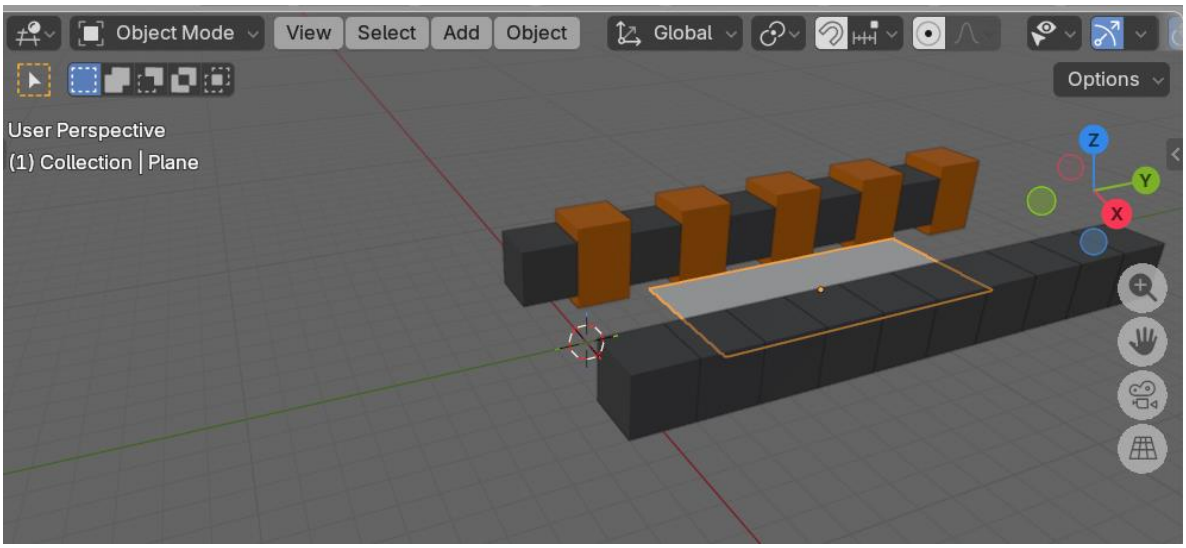
Paso 5: Iluminación Básica (Tema 4.2)

Un escenario de videojuego no está completo sin luz.

- **Lógica:** Agregar una luz de tipo "Point" para iluminar el recorrido.

```
# Añadimos una luz en el centro del pasillo
bpy.ops.object.light_add(type='POINT', location=(0, largo, 5))
luz = bpy.context.active_object
luz.data.energy = 1000  # Intensidad de la luz
```

Resultado:



Código completo:

```python
import bpy
import random

def crear_material(nombre, color_rgb):
    # Crea un material básico con un color específico
    mat = bpy.data.materials.new(name=nombre)
    mat.diffuse_color = (*color_rgb, 1.0) # RGBA
    return mat

def generar_escenario():
    # 1. Limpiar la escena previa
    bpy.ops.object.select_all(action='SELECT')
    bpy.ops.object.delete()

    # 2. Definir materiales (Basado en modelos de color RGB)
    mat_pared_a = crear_material("ParedOscura", (0.1, 0.1, 0.1))
    mat_pared_b = crear_material("ParedDetalle", (0.8, 0.2, 0.0)) # Un naranja rojizo

    # 3. Parámetros del escenario
    largo_pasillo = 10
    ancho_pasillo = 4

    # 4. Ciclo para construir paredes (Transformación: Traslación)
    for i in range(largo_pasillo):
        # Pared Izquierda
        bpy.ops.mesh.primitive_cube_add(location=(-ancho_pasillo, i * 2, 1))
        pared_izq = bpy.context.active_object

        # Aplicar material de forma alternada (Lógica de programación)
        if i % 2 == 0:
            pared_izq.data.materials.append(mat_pared_a)
        else:
            pared_izq.data.materials.append(mat_pared_b)
            # Escalamiento para darle variedad visual
            pared_izq.scale.z = 1.5

        # Pared Derecha
        bpy.ops.mesh.primitive_cube_add(location=(ancho_pasillo, i * 2, 1))
        pared_der = bpy.context.active_object
        pared_der.data.materials.append(mat_pared_a)

    # 5. Agregar un suelo (Escalamiento y Posicionamiento)
    bpy.ops.mesh.primitive_plane_add(size=1, location=(0, (largo_pasillo - 1), 0))
    suelo = bpy.context.active_object
    suelo.scale.x = ancho_pasillo + 1
    suelo.scale.y = largo_pasillo + 1

generar_escenario()
```